JavaScript常见面试题

1 JS 数据类型有哪些?

```
栈: (原始数据) string/number/boolean/null/undefined/symbol
堆: (引用数据类型)object（array 和函数属于 object）
数据类型一共 7（6 种基本类型+1 种引用类型）种
```

2介绍 JS 有哪些内置对象？

```
object 是 Javascript 中所有对象的父对象
数据封装类对象：Object、Array、Boolean、Number 和 String
其他对象：Function、Arguments、Math、Date、RegExp、Error
```

3栈与堆的区别?

```
栈与堆的储存位置不同;
    原始数据是储存在栈中简单数据段,体积小,大小固定,属于频繁使用的数据.
    引用数据类型是储存在堆中的对象,占据的空间大,如果储存在栈中会影 响运行性能,引用数据类型在栈中指明了自己的所在地.当代码解析时,会先从栈中 获取地址,然后再从堆中获取实体
```

4 js 中的作用域与变量声明提升

```
栈与堆的储存位置不同;
原始数据是储存在栈中简单数据段,体积小,大小固定,属于频繁使用的数 据
引用数据类型是储存在堆中的对象,占据的空间大,如果储存在栈中会影 响运行性能,引用数据类型在栈中指明了自己的所在地.当代码解析时,会先从栈中 获取地址,然后再从堆中获取实体;
```

5 如何转化类型?

```
转数组 parseFloat();
转字符串 toString()/string()
数组转字符串 join();
```

6 什么是面向对象编程及面向过程编程，他们的异同和优缺点

```
面向过程就是分析出解决问题所需要的步骤，然后用函数吧这些步骤一步一 步实现，使用的时候一个一个一次调用就可以了

面向对象是把构成问题事务分解成各个对象，建立对象的目的不是为了完成 一个步骤，而是为了描述某个食物在整个解决问题的步骤中的行为，面向对象是以功能来划分问题，而不是步骤
```

7 如何解释 this 在 js 中起的作用?

```
Js 中的 this,一般取决于调用这个函数的方法
1/如果函数被实例化(new 构造函数名())的情况下,this 指向全新的对象
2/如果是某标签触发什么事件,调用了这个函数,this 指向标签(整个 DOM 节 点,包含它的子元素);
3/如果函数使用了 call/apply,this 是作为参数传入对象
4/有时候 this 指向不明确的话,this 会指向 window,ES6 中的箭头函数修改 了 this 指向,永远指向作用域
```

8 说说 JS 原型和原型链

```
原型：函数都要 prototype(显示原型)属性，而 prototype 会自动初始化一 个空对象，这个对象就是原型对象
原型对象中会有一个 constructor 属性,这个属性将指向了函数本身，实例化对象都有一个_proto_(隐式原型)属性，_proto_属性指向原型 对象

原型链：从实例对象往上找构造这个实例的相关对象，然后这个关联的对象 再往上找，找到创造它的上一级的原型对象，以此类推，一直到 object.prototype 原型对象终止,原型链结束.

原型链中的原型对象中的内容,是会被不同的实例,所共有的
```

9 如何准确判断一个变量是数组类型?

```
instanceof 用于判断引用类型属于哪个构造函数的方法
```

10call 和 apply 的区别和作用?

```
apply 和 call 都是调用一个对象的一个方法，用另一个对象替换当前对象。
相同点：方法的含义是一样的，即方法功能是一样的。并且第一个参数的作 用是一样的
不同点：call 可以传入多个参数、apply 只能传入两个参数，所以其第二个 参数往往是作为数组形式传入

存在意义：实现（多重）继承
```

11 继承的方法有哪些？

```
原型链继承、构造继承、实例继承、拷贝继承、组合继承、寄生组合继承
```

12什么是闭包？闭包有什么作用？

```
由于在 js 中，变量到的作用域属于函数作用域，在函数执行后作用域会 被清除、内存也会随之被回收，但是由于闭包是建立在一个函数内部的子函 数，由于其可访问上级作用域的原因，即使上级函数执行完，作用域也不会 随之销毁，这时的子函数---也就是闭包，便拥有了访问上级作用域中的变量 权限，即使上级函数执行完后，作用域内的值也不会被销毁

闭包解决了什么：在本质上，闭包就是将函数内部和函数外部连接起来的 一座桥梁。

由于闭包可以缓存上级作用域，那么就使得函数外部打破了“函数作用域” 的束缚，可以访问函数内部的变量。以平时使用的 Ajax 成功回调为例，这里 其实就是个闭包，由于上述的特性，回调就拥有了整个上级作用域的访问和 操作能力，提高了几大的便利。开发者不用去写钩子函数来操作审计函数作 用域内部的变量了。

闭包使用场景：闭包随处可见，一个 Ajax 请求的成功回调，一个事件绑定的回调函 数，一个 setTimeout 的延时回调，或者一个函数内部返回另一个匿名函 数，这些都是闭包。简而言之，无论使用何种方式对函数类型的值进行传 递，当函数在别处被调用时都有闭包的身影

闭包的缺陷：由于闭包打破了函数作用域的束缚，导致里面的数据无 法清除销毁，当数据过大时会导致数据溢出
```

13 事件代理（事件委托）：

```
事件代理是将子元素的事件写一个父元素,让父元素代替处理,内部使用 e.target,e.target 就是触发这个事件的子元素
```

事件的各个阶段

捕获阶段 ---> 目标阶段 ---> 冒泡阶段

document ---> target 目标 ---> document

由此 addEventListener 的第三个参数设置为 true 和 false 的区别已经非 常清晰了 ，true--->代表该元素在事件的”捕获阶段”(由外向内传递)响应事件，false --->表示该元素在事件的”冒泡阶段”(由内向外传递)响应事件

14 new 操作符在创建实例的时候经历了哪几个阶段

```
new 创建了一个对象，共经历了 4 个阶段：
1. 创建一个空对象
2. 设置原型链
3. 让实例化对象中的 this 指向对象，并执行函数体
4. 判断实例化对象的返回值类型
```

15 异步编程的实现方式

```
-回调函数：
    优点：简单、容易理解
    缺点：不利于维护，代码耦合高
-事件监听（采用时间驱动模式，取决于某个事件是否发生）
    优点：容易理解，可以绑定多个事件，每个时间可以指定多个回调函数
    缺点：事件驱动型，流程不够清晰
-发布、订阅（观察者模式）类似于事件监听，但是可以通过‘消息中心’，了解现在有多少发布者， 多少订阅者
-async 函数
```

```
    优点：内置执行器、更好的语义、更广的适用性、返回的是 Promise，  结构清晰
    缺点：错误处理机制
- Promise 对象
     优点：可以利用 then 方法，进行链式写法；可以书写错误时的回调函数；
     缺点：编写和理解，相对比较难
-Generator 函数
     优点：函数体内外的数据交换、错误处理机制
     缺点：流程管理不方便
```

16 js 延迟加载的方法有哪些？

```
defer 和 async、动态创建 DOM 方式（用的最多），按需异步载入 JS
==========================================================
defer 属性：（页面 load 后执行）
    script 标签定义了 defer 属性，用途：表明脚本在执行时不会影响页面的构造。也就是所，脚本会被延 迟到整个页面解析完毕之后再执行。
    <script src=”XXX.js” defer=“defer”></script>
==============================================================
async 属性：（页面 load 前执行）
script 标签定义了 async 属性。与 defer 属性类似，都用于改变处理脚 本的行为。同样，只适用于外部脚本文件目的：不让页面等待脚本下载和执行，从而异步加载页面其他内容。异 步脚本一定会在页面 load 事件前执行。不能保证脚本会按顺序执行。
  <script src=”XXX.js” async></script>
==============================================================
动态创建 DOM 方式：
```

```
<script type="text/javascript">
   function downloadJSAtOnload() {
       varelement = document.createElement("script");
      element.src = "defer.js";
      document.body.appendChild(element);
   }
   if (window.addEventListener)
      window.addEventListener("load",downloadJSAtOnload, false);
   else if (window.attachEvent)
      window.attachEvent("onload",downloadJSAtOnload);
   else window.onload =downloadJSAtOnload;
</script>
```

17 数组从小到大排序？

```
方法一: sort 方法
var array = [1, 4, -8, -3, 6, 12, 9, 8];
function compare(val1, val2) {
    return val1 - val2;
};
array.sort(compare);
document.write(array);

方法二:冒泡排序
var array = [1, 4, -8, -3, 12, 9];
function sort(arr) {
```

```
    for(var i = 0;i < arr.length;i++){
        ////两两比较,如果前一个比后一个大,则交换位置
        for(var j = i + 1; j < arr.length; j++) {
            if(arr[i] > arr[j]) {
                var temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
            }
    }
}
sort(array);
console.log(array)

pis:求从大到小排序可以先使数组从大到小排序,然后添加 reverse()方法，使数组顺序颠倒
```

18 如何实现数组的随机排序?

```
最快是给数组添加原生sort()方法,可以随机数组,如果sort(),方法没有参数的 话,就是依照数据的
unicode 码排序的,可以在 sort()中添加一个比较函数函数:

function(a,b){
    return Math.rendom()>.5? -1:1
}
Math.rendom()在 0 到 1 之间生成一个随机数
```

19 图片懒加载

```
图片懒加载理解:
由于商城图片过多时，就会给图片加一个懒加载的缓冲效 果。当图片进入可视化区域的时候才会加载，否则
图片只是一个空标签。这样可 以优化页面渲染速度，提升用户体验。
思路:
将页面中的所有 img 属性 src 用 data-src 代替，当页面滚动至此图 片出现在可视区域时，用 js 取
到该图片的 data-src 值赋给 src。
```

20 js 中常见的内存泄漏：

```
1. 内存泄漏会导致一系列问题，比如：运行缓慢、崩溃、高延迟
2. 内存泄漏是指你用不到（访问不到）的变量，依然占据着内存空间，不能被再次利用起来
3. 意外的全局变量，这些都是不会被回收的变量（除非设置 null 或者被重 新赋值），特别是那些用来临
时存储大量信息的变量
4. 周期函数一直在运行，处理函数并不会被回收，jq 在移除节点前都会， 将事件监听移除
5. js 代码中有对 DOM 节点的引用，dom 节点被移除的时候，引用还维持
```

21 深拷贝和浅拷贝的问题：

```
1. 深拷贝和浅拷贝值针对 Object 和 Array 这样的复杂类型
2. a 和 b 指向了同一块内存，所以修改其中任意一个值，另外一个值也会随 之变化，这是浅拷贝
3. a 和 b 指向同一块内存，但是修改其中任意一个值，另外一个调用的变量， 不会受到影响，这是深拷贝
4. 浅拷贝:“Object.assign()”方法用于将所有可枚举的属性的值从一个或 多个源对象复制到目标对象，
它将返回目标对象
5. 深拷贝：JSON.parse( )和 JSON.stringify( )给了我们一个基本的解决办 法。但是函数不能被正
确处理
```

22 显示转换与隐式转换

```
JS 中有 5 中简单的数据类型（也称之为基本数据类型）：undefined、Null、 Boolean、Number、String。还有一种复杂的数据--------Object，Object 本 质是一组无序的名值对组成的。
对一个值使用 typeof 操作符可以返回该值的数据类型，typeof 操作符会返 回一些令人迷惑但技术上却正确的值，比如调用 typeof null 会返回“object”， 应为特殊值 null 被认为是一个空的对象引用。

显式转换：主要通过 JS 定义的数据转换方法
各种数据类型及其对应的转化规则：
数据类型      转换为 true 的值      转换为 false 的值
Boolean        true                  false
String      任何非空字符串            “” (空字符串)
Number     任何非零数字值(包括无穷大)      0 和 NaN
Object       任何对象                    null
Underfined     n/a                    undefined

隐式转换：是系统默认的，不需要加以声明就可以进行的转换。一般情况下， 数据的类型转换通常是由编译系统自动进行的，不需要人工干预
1. 对象和布尔值比较 对象和布尔值比较时，对象先转换为字符串，然后再转换为数字， 布尔值直接转换为数字
2. 对象和字符串比较 对象和字符串进行比较时，对象转换为字符串，然后两者进行比较
3. 对象和数字比较 对象和数字进行比较时，字符串转换为数字，二者再比较
4. 字符串和数字比较 字符串和数字进行比较时，字符串转换成数字，二者再比较，true=1， false=0
5. 字符串和布尔值比较 字符串和布尔值进行比较时，二者全部转换成数值再比较
6. 布尔值和数字比较  布尔值和数字进行比较时，布尔转换为数字，二者比较
```

23 ForEach 和 map 的区别在哪里:

```
这两个 API 都可以遍历数组
forEach 函数,是给数组中的每个都执行一遍回调函数,不会返回一个值
Map 方法是通过调用数组中的每个元素,映射到新元素中,从而床架一个新数 组
```

```
var a = [ 1,2,3];
var  doubled  =  a.forEach((num,index)=> {
    //用num和/或索引做一些事情。
});
console.log(doubled); //undefined
var c = a.map(function(num){
    return num;
});
console.log(c);  //123
```

如果是复合型类型时，如果只改变复合类型的其中某个 value 时， 将可以正常使用

24 JS 判断设备来源

```
function deviceType(){
    var ua = navigator.userAgent;
```

```
    var agent = ["Android", "iPhone", "SymbianOS", "Windows Phone", "iP ad",
"iPod"];
    for(var i=0; i<len,len = agent.length; i++){
        if(ua.indexOf(agent[i])>0){
            break;
            }
    }
}
deviceType();
window.addEventListener('resize', function(){
    deviceType();
})

//微信的 有些不太一样

function isWeixin(){
    var ua = navigator.userAgent.toLowerCase();
    if(ua.match(/MicroMessenger/i)=='micromessenger'){
        return true;
        }else{
            return false;
            }
}
```

25 ==与===的区别？

```
===为等同符，当左边与右边的值与类型都完全相等时，会返回 true；
==为等值符，用来判断值是否相同，不会判断类型是否相同
```

26 addEventListener 监听点击事件与 click 事件有什么区别?

```
addEventListener 事件可以对普通元素进行多个事件处理
click 事件只 能使元素运行最新的事件结果
```

27 null 和 undefined 的区别?

```
null 用来表示尚未存在的对象，常用来表示函数企图返回一个不存在对象
undefined 主要指定义了变量，但是并未赋值
NAN （not a Number）不是一个明确数值的数字类型
ECMAScript 认为 undefined 是从 null 派生出来的，他们的值是一样的， 但是类型却不一样
所以
null == undefined //true
null === undefined //false
```

28 如何检查对象中是否存在某个属性?

```
检查对象中存在某个属性的方式有三种:
第一种使用 in 操作符号，返回 true 或 false:
    console.log("属性名" in 对象名);
第二种使用 hasOwnProperty 方法，会返回 true 或 false:
    console.log(对象.hasOwnProperty("属性名"));
第三种是是够括号符号obj['属性名']，如果属性存在，则返回该属性的值，如 果不存在，则返回
undefined
```

29 字符串操作方法

```
splite（）：用于把一个字符串分割成字符串数组
search（）：用于减缩字符串中指定的子字符串，或检索与正则表达式相匹配的字 符串
indexOf（）：可返回某个字符换字符串中首次出现的位置
substring（）：用于提取字符串中介于两个指定下标之间的字符
trim（）：移除字符串两边的空格
replace（）：替换字符
```

30 数组操作方法。

```
length：计算数组的长度
indexOf()：返回指定元素的位置，若元素不存在返回-1
slice([par1,par2])：接受 1-2 个参数，参数对应的是要返回的是要返回项的起始位置和结束位 置，若只有一个参数，该方法返回从参数指定位置到数组结尾的所有项，如果还有两个 参数，则返回起始位置到结束位置项，但是不包括结束位置项，返回的结果是一个新数 组。
push()：向数组末位添加若干元素，返回添加元素后数组的长度
pop()：删除数组末位最后一个元素，但会被删除元素
unshift()：向数组头部添加若干元素，返回添加元素后的数组长度
shift()：删除数组头部的第一个元素，返回被删除的元素
sort()：对数组进行排序，返回排序后的数组
join(reg)：将数组中的每一个元素用指定的字符串拼接起来
reverse()：对数组中的数据进行反转，返回反转后的数组
```

31js 的 for 跟 for in 循环它们之间的区别？

```
遍历数组时的异同：for 循环 数组下标的 typeof 类型:number,
               for in 循 环数组下标的 typeof 型:string;
```

32 如何用原生 js 给一个按钮绑定两个 click 事件？

```
使用事件监听，可给一个 DOM 节点绑定多个事件（addEventListener）
```

33 拖拽会用到哪些事件？

```
dragstart---拖拽开始时在被拖拽元素上触发此事件，监听器需要设置拖拽 所需数据，操作系统拖拽文件到浏览器时不触发此事件
dragenter---拖拽鼠标进入元素时在该元素上触发，用于给拖放的元素设置 视觉反馈，如高亮
dragover---拖拽时鼠标在目标元素上移动时触发，监听器通过组织浏览器 默认行为设置元素为可拖放元素
dragleave---拖拽时鼠标移出目标元素时在目标元素上触发，此时监听器可 以取消掉前面设置的视觉效果
drag---拖拽期间在被拖拽元素上连续触发
drop---鼠标在拖放目标上释放时，在拖放目标上触发，此时监听器需要收集数据并且执行所需操作，如果是从操作系统拖放文件到浏览器，需要取消浏览 器默认行为
dragend---鼠标在拖放目标上释放时，在拖拽元素上触发，将元素从浏览 器拖放到操作系统时不会触发此事件
```

34 JS 中定时器有哪些？他们的区别及用法是什么？

```
setTimeout 只执行一次
setInterval 会一直重复执行
```

35 attribute 和 property 的区别是什么？

```
attribute 是 dom 元素在文档中作为 html 标签拥有的属性；
property 就是 dom 元素在 js 中作为对象拥有的属性；
对于 html 的标准属性来说，attribute 和 property 是同步的，是会自动更新 的，但是对于自定义的属性来说，他们是不同步的
```

36 一次性插入 1000 个 div，如何优化插入的性能

```
使用 Fragment（document.createDocumentFragment( )）
Documentfragments 是 DOM 节点，它们不是 DOM 树的一部分。通常的用例是 创建文档片段，将元素附加到文档片段，然后将文档片段附加到 DOM 树中。因为文档 片段存在于内存中，并不在 DOM 树中，所以讲子元素插入到文档片段时不会引起页面 回流。因为使用文档片段会带来更好的性能，先创建一个 div，后续的复制这个元素，避免重复创建元素，再放到元素片段里 面

var divFragment = document.createDocumentFragment（）；
let div = document.createElement（“div”）；
for（var i = 0; i <1000; i ++）{
    divFragment.append（div.cloneNode（））
    }
document.body.appendChild（divFragment）；
```

37typeof 和 instanceof 有什么区别?

```
typyof 用于判断数据的基本类型
instanceof 用于判断一个变量是够属于某个对象的实例。也可以用来判断某个 构造函数的 prototype 属性是否存在另外一个要检测对象的原型链上
```

38 简述同步和异步的区别?

```
同步:就是指一个进程在执行某个请求的时候，若该请求需要一段时间才能返回信 息，那么这个进程将会一直等待下去，直到返回信息之后才继续执行后面的代码

异步:是指进程不需要一直等下去，而是继续执行下面的操作，不管其他进程的状 态。当有消息返回时系统会通知进程进行处理，这样可以提高执行的效率。
```

39 如何阻止事件冒泡和默认事件？

```
IE:
cancelBunnle（IE）
return false;

event.preventDefault( );//阻止默认事件
event.stopPropagation( );//阻止冒泡
```